Infrathrone Week-2 Interview Playbook

PLAYBOOK STRUCTURE

Each section includes:

1. **The Interviewer's Intent** (what they are testing)
2. **The Question**
3. **The InfraThrone Answer Framework**
4. **Sample 10/10 Answer**
5. **Red Flags (0/10 answers)**
6. **Follow-up Traps + How to Win Them**

SECTION 1: Kubernetes Control-Plane Deep Diagnostics

Topics Covered:

API Server, etcd, Scheduler, Controller Manager, Kubelet, CRI, Networking, Node lifecycle.

QUESTION 1 — "kubectl is hanging. How do you debug this?"

Interviewer Intent

Checks your **Layer-2 thinking**: API server health, network reachability, client → server flow.

InfraThrone Answer Framework

Use the InfraThrone Ladder:

Layer 1 → Confirm user-side issue
Layer 2 → API server health (/livez, /readyz)
Layer 3 → etcd latency
Layer 5 → Node load & kubelet logs
Layer 6 → Network path

Sample 10/10 Answer

"If kubectl hangs, I don't touch workloads.
First, I target the API Server.

I run: curl -k https://127.0.0.1:6443/livez
If that hangs → API Server blocked.

Next, I check etcd latency: etcdctl endpoint health

90% of the time, the root cause is etcd overload (high fsync, no leader, disk pressure).

Only after confirming control-plane health do I move to kubelet or CNI."

Red Flags (automatic reject)

- “I restart kubelet.”
- “I check pods first.”
- “Maybe cluster is down.”
- “I re-deploy API server.”

Follow-up Trap

Interviewer: “kubectl get nodes works, but kubectl get pods -A hangs. Why?”

Winning answer: API Server talking to etcd with high latency → pods live in etcd; nodes come from node heartbeats.

QUESTION 2: “How do you detect and fix etcd slowness?”

InfraThrone Framework

Use **Layer-3: etcd → Data Path Debugging**

Commands

```
ETCDCTL_API=3 etcdctl endpoint status --write-out=table
kubectl get events --sort-by=.metadata.creationTimestamp | tail
iostat -xm 1
df -h
```


Perfect Answer

“etcd slowness manifests as:

- API server 5xx
- Timeouts
- Scheduler lag

I confirm by checking: etcdctl endpoint health → timeouts
Then I check disk IOPS and fsync latency. 90% of real outages come from disk pressure.”

Follow-up Trap

“etcd is healthy, but API server is slow.”

Check admission controllers, API server QPS, node informers.

SECTION 2: Scheduler / Controller Manager Debugging

QUESTION 3: “Pods stuck in Pending. Debug?”

InfraThrone Answer Framework

Layer-4:

- Check node taints
- Check scheduler logs
- Check resource requests vs allocatable
- Check CSI volume issues
- Check anti-affinity rules

Perfect Answer

“Pending means scheduler cannot match pod → node.
I run: kubectl describe pod
And check the SchedulingEvents.

Common root causes:

- Taints
- Insufficient CPU/memory
- Inter-pod affinity
- CSI volume attach failures.”

Follow-up Trap

“What if only statefulsets are stuck?”

Storage layer issue (PV/PVC mismatch, multi-attach errors).

SECTION 3: Kubelet & Node Lifecycle

QUESTION 4: “Node went NotReady. How do you debug?”

InfraThrone Framework

Layer-5:

Commands

- kubectl describe node
- journalctl -u kubelet -f
- crictl ps
- systemctl restart containerd

Root Causes You Must Mention

- Kubelet cannot talk to API Server
- Container runtime broken
- DiskPressure or MemoryPressure
- CNI crashed

Perfect Answer

“If node is NotReady: I check heartbeat, Kubelet logs, container runtime, CNI availability.

Most common causes: DiskPressure and containerd deadlocks.”

SECTION 4: Networking (CNI, kube-proxy, iptables)

QUESTION 5: “Service reachable from inside cluster but not outside. Debug?”

InfraThrone Framework

Layer-6:

- Check kube-proxy
- Check NodePort / firewall rules
- Check endpoint list
- Check CNI routing

Perfect Answer

> “Inside cluster works → CNI OK.
> Outside fails → NodePort load-balancer path.
>
> I check iptables for kube-proxy and external LB health.”

SECTION 5: Workload Debugging (Layer-7)

QUESTION 6: “Why is my pod CrashLoopBackOff?”

InfraThrone Framework

Always answer **in layers**:

- Layer 7 → app crash
- Layer 5 → node resource issue
- Layer 4 → controller repeatedly restarting
- Layer 3 → high etcd latency delaying readiness

Perfect Answer

> “CrashLoop means the container exits repeatedly.
> Most engineers debug the container only, but I also check node pressure and readiness delays.”

SECTION 6: Cross-Layer Outage Scenarios

QUESTION 7: “API Server is up. etcd is healthy. But 30% of requests 500. Debug?”

Perfect Answer

“This is classic Thread Pool Exhaustion.
When request latencies spike, API server worker threads saturate.

I check: /metrics → apiserver_request_total
Admission webhooks
Mutating/Validating policies
Large CRDs causing informer explosions.”

QUESTION 8: “DNS working, ping working, but curl failing.”

InfraThrone Winning Answer

Means ICMP + UDP working

TCP handshake broken

Likely CNI, kube-proxy, conntrack exhaustion, eBPF corruption

---

SECTION 7: Behavioral & Leadership Questions

Q9: “Explain a Kubernetes outage you solved.”

Your answer must follow the **STAR-K8s Format**.

S: Situation: Cluster experiencing API server timeouts.

T: Task: Restore cluster without downtime.

A: Action: Describe using the **7-layer framework**.

R: Result: Restored cluster in 12 minutes.

K8s Layer Tie-in: Mention which layer you diagnosed (etcd, scheduler, kubelet).
